

THE VAULT

Annotations, citations, answer keys and technical justification for every task in the session — the scenario, the programming, and the quantum computing underneath it.

Prepared for internal training delivery · Not for learner distribution

THE VAULT — Facilitator & Admin Guide

A companion reference for whoever is running the session — not for the learner. Covers the scenario, all four tasks, the exact answer keys, the programming and quantum concepts underneath each one, why the activity is built the way it is, and where every factual claim on screen comes from.

0 · How to use this guide

The Vault (`vault.html`) is a single continuous story: a learner wakes on a concrete floor, is talked through a heist by a voice on the phone (Vale), and in about twenty minutes writes four real programming constructs and runs one real quantum search algorithm — with the actual arithmetic, not a simulation of it. Nobody touching the learner-facing page needs to know any of this. This guide is for you: to answer the question that comes after "how does that actually work," to know exactly which answers are right and why the wrong ones are wrong, and to have a citation ready if someone technical in the room pushes back.

Structure: one section per task, in the order the learner meets them. Each section covers what's on screen, the concept it teaches, the full answer key, and the sourced explanation. A glossary, a quiz answer key, delivery notes, and the full reference list are at the end.

A word on tone: nothing here is designed to make anyone feel behind. The whole point of the closing screen is "you are not behind — almost nobody is ahead." Keep that register when you're asked questions live.

1 · Why it's built this way

Three design decisions, each with a reason behind it.

The story is not decoration; it's the delivery mechanism. A heist with a six-minute clock is what makes "you need somewhere to keep that number" feel like a real problem instead of a definition to memorise. This follows two established ideas in learning science: **cognitive load theory** — working memory is a scarce, easily-overloaded resource, so instruction should minimise effort spent on anything other than the concept itself¹ — and **anchored instruction / situated learning** — people learn abstract ideas faster and retain them longer when the idea is embedded in a concrete, meaningful problem rather than presented as a decontextualised fact². Separate, more recent work on narrative-led instructional video found measurably better short-term retention and transfer than lecture-style delivery of the same content³ — that study wasn't run on adult career-changers learning to code, so treat it as supporting evidence for the *shape* of the design, not proof this exact audience will behave identically.

Every blank is filled by choosing, never by dragging or typing. This matters because the wrong choices are real wrong choices with real consequences on screen — picking `x` as a variable name still runs, but the activity tells you why nobody will thank you for it in three weeks. This is the **worked-example effect**: novices learn a new procedure better from studying correct (and incorrect) worked steps than from unconstrained trial and error, because it removes the extraneous load of search while keeping the load that actually matters — deciding what's right⁴. It's also why every gate in the build (the Valid/Invalid buttons, the "run it" button, the next-room button) stays visibly and honestly locked until the real condition is met, rather than looking clickable and silently doing nothing — a gate that lies about being open is worse than no gate at all, because it reads as broken software rather than a task not yet finished.

Nobody can be hard-locked out by the clock. Okonjo buys the learner ninety seconds once, with a line of dialogue, if the six minutes run out — the clock never actually ends the session. A tool aimed at absolute beginners who have described themselves as anxious about technology cannot have a failure state that ends the activity; the tension is motivational, not punitive.

2 · The scenario — Act 0, and the clock

On screen: the learner wakes on a concrete floor in "sub-basement three." Vale, on the phone, explains the job: a vault floor with three rooms, and a ledger in the last one. The learner says "I have never written a line of code in my life." Vale: "*You will have, in about twenty minutes. Touch that door and the alarm starts. Six minutes from then.*"

The mechanics, for your reference: - The clock is 360 seconds (`LIMIT = 360`), and it only starts once "Touch the door" is pressed. - Every action costs seconds (`cost (sec)`): opening a box by hand costs 4s; running a search step costs 8s; a wrong variable-decode guess costs 8s; a missed room attempt costs 20s; each account read in the ledger's classical scan costs roughly 1s per account. - If the clock would go negative, Okonjo (a character named but never seen) buys ninety seconds once, with a toast notification: "*Okonjo bought you a minute. Do not need another.*" The clock is allowed to run into negative time after that — the debrief screen just says "Out. Just." instead of "Out, with [time] to spare."

Facilitator note: the six-minute figure is a narrative device to create urgency, not a claim about how long the *real* activity takes. A facilitated run with discussion, questions, and letting people finish typing rather than racing typically runs much longer than six minutes of wall-clock time — see §9 for realistic session timing.

3 · Task 1 — The Outer Door

Teaches: variable, until loop. Wrapped in: a shift cipher.

What's on screen

The intercepted combination reads **CHBSA**. Vale explains it's a shift cipher — every letter moved along the alphabet by the same amount — and that the shift number is "the whole secret." The learner first names a box and puts a number in it (`DECODER.PY : <name> = <number>`) then `decode("CHBSA", <name> )`), then writes a loop to crack a second, harder door with no hint at all.

The concept: variable

A variable is a named location that holds a value you're allowed to change. That's the entire definition — there's no deeper trick to it. The pedagogical point the activity is making is about **naming**, not mechanics: `x = 7` and `shift = 7` behave identically to the computer and differently to the human who reads the code three weeks later. This is a completely standard point in software engineering education; no external citation is needed for the definition itself, only for the *choice* to teach it this way (§1).

Answer key — the decoder

- **Name the box** — any of `shift` , `x` , `thing` , `shift_amount` is accepted and the program runs. `shift` and `shift_amount` are marked **correct** (green). `x` and `thing` are marked **weak** (amber) and trigger an explanatory beat: *"It runs. But in a month, nobody reading x knows what x is."* — the activity is deliberately letting a technically-working but badly-named answer through, then explaining why it's still wrong in the way that matters.
- **Put a number in it** — the options are 3, 5, 7, 11, 19. **Only 7 is correct.** The cipher is `VAULT` shifted by 7 to give `CHBSA` ; entering any other number decodes to gibberish and costs 8 seconds, with the beat *"Wrong number. The letters moved, but not into a word. Try another."* There's no way to "guess around" this — the learner has to actually try values until the decode succeeds, which is the entire point of the next screen (the loop).

Answer key — the until loop

Second door, secret word `LEDGER` , shifted by 14. The code frame is: `shift = 1 until <test>: try_it(shift) shift = shift + <step>` Two blanks, three options each. **Only one full combination actually opens the door when run** — the builder will let you *select* any option (that's the "code as thought" mechanic: a wrong pick still fills the blank so you can run it and see what it actually does), but only one selection makes the loop work:

Blank	Options offered	What happens if chosen	Correct?
Stop test	<code>it_is_a_word</code>	Loop runs until the decoded text is a real word — stops exactly on success	✓ Correct
	<code>shift == 25</code>	Runs all 25 shifts regardless of whether one worked, then stops anyway — <i>"stopped at 25. Did it work? The loop has no idea."</i>	✗ Runs out instead of succeeding
	<code>never</code>	No stop condition at all — infinite loop, demonstrated live in the log	✗ Never terminates
Step size	1	Checks <code>shift = 1, 2, 3 ...</code> in order — guaranteed to land exactly on 14	✓ Correct
	0	<code>shift</code> never changes, so the test never changes either — same failure as <code>never</code>	✗ Infinite loop (dead counter)
	5	Checks 1, 6, 11, 16, 21 — skips straight past 14 — <i>"stepped straight past it. 5 at a time misses shifts."</i>	✗ Overshoots the target

The only working pair is `it_is_a_word + 1` . This is a genuinely good test of understanding: a learner who picks `shift == 25` because it "sounds like it should work" (it superficially resembles a valid stopping condition) will see it fail for a reason that's actually instructive — a loop that counts down to zero isn't the same as a loop that checks whether it succeeded.

Technical accuracy note

A shift (Caesar) cipher is a teaching prop, not real cryptography — it has only 25 possible keys and was broken in antiquity. Nothing in this task claims otherwise; the "real encryption" claim is reserved for the closing screen (§7), where it's about modern public-key cryptography, a different and much harder problem. Don't let "the loop cracked a cipher in a few tries" imply anything about how hard breaking a bank's actual encryption would be — that's addressed explicitly later and the two should stay separate in your own explanations too.

4 · Task 2 — Room One

Teaches: sequence / linear search. Introduces: superposition, measurement, interference, the first real Grover step.

What's on screen

Four boxes, one key, opened by hand — a linear search, one box at a time, 4-second cost each. Then the same four boxes are put into **superposition** ("spinning"), followed by one round of **Mark + Boost**, after which opening one box succeeds with certainty.

The concept: linear search (classical)

Opening boxes one at a time, from the top, until you find the one you want, is a **linear search** — literally what every conventional computer does to find an unindexed item, billions of times a second. Worst case for four boxes is four opens; the activity's own beat names this directly: *"Four boxes is nothing. The last room has sixty-four, and the clock is running."*

The concept: superposition and measurement

A qubit can be placed in a state that represents a weighted mix of 0 and 1 simultaneously; **measuring it forces a single, definite classical outcome**, collapsing that mixed state.⁵ The activity's own analogy — a spinning coin that is "genuinely both" while it spins, and has to land on one face the instant you look — is a fair, standard way to describe this to a lay audience, provided you don't push it further than that. Two care points if you're asked to elaborate:

- Don't say a qubit "is" both 0 and 1. Better: it has a *probability* of being found in either state until measured.
- The activity is explicit that spinning all four boxes "is lovely, and useless" on its own — grabbing one from superposition still gives one-in-four odds, exactly as if you'd guessed. **Superposition alone does not search anything.** This is deliberate and correct, and it sets up the actual mechanism in the next screen.

The concept: interference (Mark + Boost) — the actual mechanism

This is the single most important thing to get right if anyone in the room has technical background, because the most common popular explanation of quantum computing — "it tries every answer at once" — is flatly wrong, and this activity is careful not to teach it. The Vault's own note addresses the question directly: *"But how does Mark know which one? It does not. The lock recognises its own key when a key is tried — it can say 'that one' without telling you which one it was. Marking is a test, not knowledge."*

The real mechanism is **amplitude amplification via interference**.⁶⁷

1. **Mark** flags the correct answer by flipping the sign (phase) of its probability amplitude — everything else is untouched.
2. **Boost** ("reflect everything about the average") redistributes amplitude so that the flipped, marked answer's amplitude grows and every other answer's amplitude shrinks.
3. Repeat, and the correct answer's probability of being read out on measurement climbs toward certainty.

Superposition creates many possibilities at once; **interference is what steers the measurement toward the right one**. Simply superposing every answer and measuring immediately gives a *uniformly random* answer, not the correct one — which is exactly why "spin them and grab one" in the previous screen still gave 25% odds. The wave-cancellation scope on screen (two identical waves; click to flip one, and they flatten to nothing) is a faithful physical picture of destructive interference, not a metaphor invented for this activity.

The real numbers (verified against the live build's own maths)

For N = 4 boxes, one Mark + Boost step gives **exactly 100%** — this isn't rounding, $\sin^2(3 \times \arcsin(\frac{1}{2})) = \sin^2(90^\circ) = 1$ exactly. Press it a second time and the odds *fall* to 25% ($\sin^2(5 \times 30^\circ) = \sin^2(150^\circ) = 0.25$) — the beat *"Press it again and watch the odds fall. There is a right number of steps, and more is not better"* is stating a real, checkable property of the algorithm, not a game-balance choice. This periodic overshoot is the same mechanism explored properly in Task 3.

5 · Task 3 — Rooms Two and Three

Teaches: counted loop. Deepens: Grover's algorithm at scale, the learner now chooses the step count themselves.

What's on screen

Room Two: 16 boxes. Room Three: 64 boxes. The code frame is now a **counted loop**: `all_boxes.spin() repeat <how many> times: mark() boost() look()` The learner picks the repeat count from a short list, runs it, and either finds the key or misses and has to adjust.

The concept: counted loop, and why it matters here specifically

A counted loop runs a fixed block of instructions a stated number of times without the programmer writing it out longhand. The flashback that precedes this task makes the case concretely: six lines of `mark(); boost();` repeated three times becomes two lines — *"and when the room turns out to need six instead of three, you change one character rather than rewriting the lot."* This is the first time in the story the learner has needed to specify *how many* repetitions rather than *until what condition* — a deliberate contrast with Task 1's until loop.

Answer key — the correct step counts

The activity computes the true optimum live, using the closed-form estimate of Grover's optimal iteration count, $k \approx$

`round((pi/4)/asin(1/sqrt(N)) - 0.5)` — not a brute-force search over possibilities, which (as documented in the build history) can find a later, coincidentally-higher periodic peak and give a worse answer.

Room	N	Options offered	Best k	Odds at best k
Room Two	16	1, 2, 3, 5	3	96.1%
Room Three	64	2, 4, 6, 12	6	99.7%

(Both figures independently recomputed and confirmed for this guide.) Picking fewer than the best k under-amplifies (odds climbed, "but not far enough to bet the night on"); picking more **overshoots and the odds swing back down** — genuinely, not as a quirk of the demo, for the same periodic reason as Task 2's second press. There is a real right answer, and it is not "more steps is always safer."

Classical vs quantum, stated precisely

Worst-case classical (open every box by hand) is **N** — 16 or 64. Grover's algorithm needs on the order of $\sqrt{N}$ oracle calls — 4 and 8 respectively, and the activity's own numbers (3 and 6 steps) sit right around that scaling, which is the correct headline comparison to give a non-technical audience.⁶ The **average** classical case (stop as soon as you find it, not always at the end) is N/2 — 8 and 32. Use whichever framing suits the question you're asked, but don't quote "500,000 vs 1,000" (a figure from an earlier draft, for a different, million-item example) in the context of this build's own 16- and 64-box numbers — keep the numbers attached to the room they came from.

Important caveat to have ready: this is a *quadratic* speedup, not an exponential one, and it is *provably optimal* — no quantum algorithm can search an unstructured list faster than roughly $\sqrt{N}$ queries.⁸ "Quadratic" sounds modest next to popular quantum-computing claims; it is genuinely what the published mathematics says, and the honest framing is part of what makes this activity trustworthy rather than hype.

6 · Task 4 — The Ledger

Teaches: conditional. Closes the loop on: the oracle as a concept — the learner's own `if` rule becomes the thing the quantum machine searches for.

What's on screen

Sixty-four accounts; exactly one breaks a rule (moves more than £10,000 **and** is not in the UK). The learner writes the rule, runs it as a classical scan (one account at a time), then hands the *same* rule to a quantum search.

The concept: conditional, and the two bugs it's built to catch

```
for account in ledger: if amount <op1> <lim> and country <op2> <ctry>: flag(account)
```

Full answer key:

Slot	Correct	Why the wrong options are wrong
op1 (first comparison)	>	< finds small movements, the rule wants large ones. == matches only exactly £10,000 — the rule says <i>more than</i> .
lim (amount)	10000	100 — every account in the book moves more than £100, so nothing useful gets flagged. 1000000 — set so high nothing is ever flagged.
op2 (second comparison)	!=	= assigns a value into something; it does not ask a question at all, and can't legally sit inside an <code>if</code> test the way the learner has written it. This is the single most common real-world beginner bug in conditional logic.
ctry (country)	"UK" (quoted)	UK without quotes is not text — it's a reference to a <i>variable</i> called <code>UK</code> , which was never created, so the program would look for something that doesn't exist. "10000" is the number restated as text, and answers the wrong question entirely.

Both traps in this task — `=` vs `!=`, and missing quote marks around text — are, respectively, the most common logic bug and the most common syntax bug that real beginner programmers produce in their first weeks with conditionals. Teaching both inside one meaningful rule (rather than as two abstract syntax rules) is why this task earns its place as the series' fourth and final construct.

The real numbers

Classical scan reads accounts one at a time until the flagged one turns up — **up to 64 reads**, and the demo shows however many it actually took *this* run (the guilty account's position is randomised each time). The quantum scan runs the identical rule as an oracle for **6 steps** (the same `bestK(64) = 6` computed in Task 3) at **99.7%** confidence.

The concept: the oracle, made literal

This is the idea the whole build is constructed to deliver, and it's worth reproducing the activity's own words because they're doing real technical work, not just narrative flourish: "You did not learn a second language. You wrote one rule, and handed it to a different machine... Your `if` became the Mark. That is all an oracle is: a rule that recognises the answer when it sees it."

Technically: a Grover **oracle** is a subroutine that flips the phase of any state satisfying a given predicate, leaving every other state untouched — which is exactly the boolean condition the learner already wrote (`amount > 10000 and country != "UK"`), just realised as a phase-flip instead of a `True / False` return value.⁶ The pedagogical claim "your conditional is the oracle" is not a simplification for beginners — it is the literally correct relationship between classical boolean predicates and Grover oracles.

Important technical clarification for you, not the learner

Grover's algorithm and "harvest now, decrypt later" are not the same threat, and it's worth knowing the difference even though the activity (correctly, for its level) blurs them for narrative pace. Grover offers a *quadratic* speedup against unstructured search / brute-force problems — the practical consequence is that symmetric-key algorithms (like AES) need roughly double their key length to keep the same security margin against a quantum attacker, which is a known, manageable mitigation. The far bigger real-world threat driving "harvest now, decrypt later" is a *different* quantum algorithm, **Shor's algorithm**, which gives an *exponential* speedup at factoring large numbers and computing discrete logarithms — the exact hard problems that today's public-key cryptography (RSA, Diffie-Hellman, elliptic-curve cryptography) relies on. Shor's algorithm is not taught or referenced anywhere in this build. If a technically literate person asks "is this Shor's algorithm?" — no, this activity teaches Grover; the *reason organisations are migrating cryptography right now* is mostly about Shor, but the two-part warning ("your data is exposed to a future machine that doesn't exist yet") is correct regardless of which algorithm eventually does the breaking.

"Save now, decrypt later" — the sourced version

The closing screen's line is: "Real encryption would take every computer on Earth longer than the universe has existed — today. Which is why intelligence agencies are recording encrypted traffic right now and storing it, to open when the machine arrives." This is a real, actively-discussed risk, not narrative invention:

- The concept and its stakes for institutions with long data-confidentiality requirements (exactly the finance-sector framing here) are examined in a 2025 Federal Reserve working paper on post-quantum migration.⁹
- NIST finalised the first three post-quantum cryptography standards — **FIPS 203 (ML-KEM)**, **FIPS 204 (ML-DSA)**, and **FIPS 205 (SLH-DSA)** — in August 2024, specifically to give organisations something to migrate to.¹⁰
- The US NSA's **CNSA 2.0** suite sets a concrete migration timeline for national security systems: no mandatory enforcement before the end of 2025, all new system acquisitions compliant from 2027, legacy systems phased out by 2030, and full compliance required by 2031.¹¹ This is a US national-security-system timeline, not a UK financial-services requirement — cite it as evidence of how seriously the risk is being taken, not as a rule that applies to your organisation.

7 · The close — Act 5

What's on screen: a debrief naming all six constructs taught (variable, until loop, counted loop, conditional, superposition, interference), the session's own classical-vs-quantum tally (a clean run finishes around 148 hand reads against 16 search steps), the harvest-now-decrypt-later note above, a UK quantum workforce statistic, and a five-question quiz.

The UK quantum statistic — read this before anyone asks

The on-screen line is: "About 3,000 people work in quantum in the UK. The sector is aiming for 100,000-plus jobs over the next two decades... Source: UK Quantum Skills Taskforce / NQCC." Be aware there is a real discrepancy worth knowing about rather than being caught by:

- The exact "**~3,000 now / 100,000+ target**" pairing traces to a March 2026 blog post by the NQCC's Director, published on the NQCC's own site.¹² It's a legitimate, attributable statement from the UK's national quantum computing body — but it's a blog post, not an independently audited figure, and the post itself gives no sourcing for either number.
- The more rigorously sourced **UK Quantum Skills Taskforce report** (a formal government publication, first published May 2025) gives a different current figure — **around 1,700** people, based on 2022-23 HMRC data — and does **not** set a UK-specific 100,000-job target at all; it instead cites *global* market projections of 250,000 new quantum-computing jobs worldwide by 2030 and 840,000 by 2035.¹³

If pressed: both figures are genuine and both come from credible UK sources; they're measuring slightly different things (and one is more formally audited than the other) rather than one being "wrong." The honest answer is "a UK national body puts today's figure at around three thousand, government's own labour-market data puts it nearer seventeen hundred, and either way it's a very small, very young field with enormous room to grow" — which if anything strengthens the activity's actual point.

Quiz answer key

The activity's own five questions and explanations, reproduced here for convenience — all are already answer-locked and explained on screen, so this is purely for your reference:

1. `shift = 7`. **What is `shift`?** → A variable — a named box holding a value you can change. Not a loop or a function.

2. **Trying keys with no idea how many it will take — which loop?** → *An until loop.* A counted loop either runs out early or keeps going after it's already won.
 3. **Why doesn't `country = "UK"` belong inside an `if`?** → *A single `=` assigns a value; it doesn't ask a question.* (Not "it should be `!=` instead of `=`" as a standalone fix — the *reason* is the assignment/ comparison distinction, which the correct answer states explicitly.)
 4. **64 boxes, 6 search steps — why so few?** → *Each step tips the odds, and it takes about $\sqrt{64}$ of them.* Not "quantum computers are just faster," and explicitly not "it checks every box at the same time and returns the answer" — that's the myth this whole guide has been careful to rule out.
 5. **The ledger rule found the account in 6 steps instead of 64 reads — what changed?** → *Nothing about the rule — the same condition became the Mark.* Not "rewritten in a quantum language," and not "the rule got simpler."
-

8 · Glossary

Programming terms

- **Sequence** — instructions carried out in the order they're written; the most basic property any program has.
- **Variable** — a named location holding a value that can change; the name is for the next human reader, not the computer.
- **Loop** — repeating a block of instructions. Two kinds appear in this build: a **counted loop** (repeat a known number of times) and an **until loop** (keep going until a condition is met — used when you don't know the number in advance).
- **Conditional** — a question the program asks about one piece of data, branching what happens next on the answer (`if ... :`).
- **Comparison vs assignment** — `=` assigns a value into something; `==` and `!=` ask whether two things are equal or different. Confusing the two is the single most common beginner conditional bug, and is deliberately taught via a wrong-answer path in Task 4.

Quantum terms

- **Qubit** — the quantum unit of information; unlike a classical bit, it can be placed in a superposition of 0 and 1.
 - **Superposition** — a state representing a weighted mix of outcomes at once; it is not a hidden, predetermined answer.
 - **Measurement / collapse** — observing a qubit forces one definite classical outcome, ending the superposition.
 - **Amplitude** — the quantum-mechanical quantity (not a plain probability; it can effectively be "positive" or "negative") whose square gives the probability of an outcome on measurement. Interference works by combining amplitudes, not probabilities directly.
 - **Interference (constructive / destructive)** — amplitudes from different paths combining to reinforce or cancel. Destructive interference cancelling the "wrong answer" amplitudes, and constructive interference reinforcing the "right answer" amplitude, is the actual mechanism behind Grover's speedup — not parallel trial of every answer.
 - **Oracle** — in Grover's algorithm, a subroutine that flips the phase of any state satisfying a given condition. A classical boolean test (an `if`) and a Grover oracle are, formally, the same kind of object realised two different ways.
 - **Grover's algorithm** — a quantum search algorithm giving a *quadratic* speedup for unstructured search: roughly $\sqrt{N}$ oracle calls instead of a classical N (worst case) or $N/2$ (average case). Proven optimal — no faster quantum algorithm for this problem exists.
 - **Quadratic speedup** — the algorithm's headline property: time scales with the square root of the classical time, not a fixed multiple and not an exponential improvement.
-

9 · Facilitator delivery notes

- **Realistic timing.** The on-screen clock (six minutes) is a narrative device. A facilitated run — reading dialogue aloud, letting people actually think through each picker rather than guessing, pausing at each flashback, and running the quiz at the end — comfortably takes 20–35 minutes for one person working through it live, and longer for a group pausing to discuss. Budget accordingly; don't let the on-screen clock set your session's actual pace.
- **The gates are supposed to look locked when they're locked.** If a learner says a button "isn't doing anything," check whether the relevant builder above it is actually complete — every gate in this build (Valid/Invalid buttons, Run buttons, Next-room buttons) is genuinely disabled, with a lock note, until its condition is met. This was a real bug in an earlier version (buttons that *looked* live and silently refused) and has been fixed throughout — but it's worth knowing what "locked" looks like so you can reassure someone it's working as intended, not broken.
- **"I picked an option and it didn't work" is a feature, not a bug.** In Task 1's until-loop and Task 3's step-count screens, the picker will accept and display *any* offered option — filling the blank doesn't mean it's the functionally correct choice, only that it's a legal thing to put there. Whether it actually works is only revealed when the learner presses Run. This is deliberate: it's what "code as thought, not clicks" means in practice, and it's worth naming explicitly if someone is confused about why their selection "counted" but still failed.
- **The most common conceptual sticking point is "why isn't more always better?"** (Task 2's second Mark+Boost press, and Task 3's step-count overshoot). Have the exact numbers in §4–5 ready — they're genuinely counterintuitive and genuinely correct, and showing the real percentages lands better than describing the effect in words alone.
- **The second most common sticking point is the `=` / `==` / `!=` distinction** in Task 4. It's the most common real beginner bug in the world, not a sign anyone is struggling unusually — say so.
- **Accessibility.** Every interactive element is click/tap-based — there is no drag-and-drop to navigate — and the build respects `prefers-reduced-motion`, disabling animation for anyone whose system is set that way. No external fonts or scripts load, so it runs on

- locked-down machines.
- **If someone wants to go deeper**, the four flashbacks (triggered automatically between tasks) each contain a small worked code example independent of the heist narrative — they're the fastest way to show someone the underlying idea in isolation if the story framing isn't landing for them personally.

10 · References

1. Sweller, J. (1988). "Cognitive load during problem solving: Effects on learning." *Cognitive Science*, 12(2), 257–285. [↵](#)
2. Cognition and Technology Group at Vanderbilt (1990). "Anchored Instruction and Its Relationship to Situated Cognition." *Educational Researcher*, 19(6), 2–10. See also: Lave, J., & Wenger, E. (1991). *Situated Learning: Legitimate Peripheral Participation*. Cambridge University Press. [↵](#)
3. Ginting, D., Woods, R. M., Barella, Y., Limanta, L. S., Madkur, A., & How, H. E. (2024). "The Effects of Digital Storytelling on the Retention and Transferability of Student Knowledge." *SAGE Open*. (Study population: n=41 pre-service teachers, early twenties, Indonesian university — general learning-science evidence for narrative framing, not programming-specific or age-matched to this activity's audience.) [↵](#)
4. Sweller, J., & Cooper, G. A. (1985). "The Use of Worked Examples as a Substitute for Problem Solving in Learning Algebra." *Cognition and Instruction*, 2(1), 59–89. (Original domain is algebra; extrapolated here to programming instruction.) [↵](#)
5. IBM, "What is a qubit?" [ibm.com/think](https://www.ibm.com/think), Schneider, J. & Smalley, I. (2024, updated 2026). <https://www.ibm.com/think/topics/qubit> — see also NIST, "What Is Superposition?" <https://www.nist.gov/video/what-superposition> [↵](#)
6. IBM Quantum Learning, "Grover's algorithm" module. <https://quantum.cloud.ibm.com/learning/en/modules/computer-science/grovers> [↵↵↵](#)
7. Pavlus, J. "Why Is Quantum Computing So Hard to Explain?" *Quanta Magazine*, June 8, 2021 (quoting Scott Aaronson on the "central fallacy" of popular quantum-computing explanations). <https://www.quantamagazine.org/why-is-quantum-computing-so-hard-to-explain-20210608/> [↵](#)
8. Grover, L. K. (1996). "A fast quantum mechanical algorithm for database search." *Proceedings of the 28th Annual ACM Symposium on Theory of Computing* (STOC '96), 212–219. Preprint: <https://arxiv.org/abs/quant-ph/9605043> — optimality of the $\sqrt{N}$ bound established in Bennett, C. H., Bernstein, E., Brassard, G., & Vazirani, U. (1997), "Strengths and Weaknesses of Quantum Computing," *SIAM Journal on Computing*, 26(5). [↵](#)
9. Federal Reserve. "Harvest Now, Decrypt Later": Examining Post-Quantum [Cryptography Migration]." FEDS Working Paper, 2025. <https://www.federalreserve.gov/econres/feds/files/2025093pap.pdf> [↵](#)
10. NIST. "NIST Releases First 3 Finalized Post-Quantum Encryption Standards." Aug 13, 2024. <https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards> (FIPS 203 / ML-KEM, FIPS 204 / ML-DSA, FIPS 205 / SLH-DSA.) [↵](#)
11. NSA. "Commercial National Security Algorithm Suite 2.0" FAQ. https://media.defense.gov/2022/Sep/07/2003071836/-1/-1/0/CSI_CNSA_2.0_FAQ_.PDF — applies to US National Security Systems, cited here as evidence of migration urgency, not as a UK compliance requirement. [↵](#)
12. Cuthbert, M. (Director, National Quantum Computing Centre). "The UK has thrown down the gauntlet on quantum." NQCC blog, March 20, 2026. <https://www.nqcc.ac.uk/blogs/the-uk-has-thrown-down-the-gauntlet-on-quantum/> [↵](#)
13. UK Quantum Skills Taskforce Report. [gov.uk](https://www.gov.uk), first published May 13, 2025 (updated June 24, 2025). <https://www.gov.uk/government/publications/uk-quantum-skills-taskforce-report/uk-quantum-skills-taskforce-report> [↵](#)